

UDP Client Code Explanation and Viva Questions

This document provides a detailed, block-by-block explanation of the provided UDP client code, along with a comprehensive set of viva questions and answers to help you confidently present your Computer Networks mini-project.

1. Code Explanation: `server.c` (UDP Client)

The provided C code implements a basic UDP (User Datagram Protocol) client. This client is designed to send log messages entered by the user to a specified server. It continuously prompts the user for input until the user types "exit".

Let's break down the code into logical blocks:

1.1. Header Files and Preprocessor Directives

Plain Text

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 5000
#define SERVER_IP "127.0.0.1"
#define BUFFER_SIZE 1024
```

- **#include <stdio.h>** : This header file provides standard input/output functions like `printf()` for printing to the console and `fgets()` for reading input.
- **#include <stdlib.h>** : This header provides general utility functions, including `exit()` for program termination.
- **#include <string.h>** : This header provides string manipulation functions such as `memset()` for clearing memory, `strlen()` for getting string length, and `strcmp()` for string comparison.
- **#include <unistd.h>** : This header provides access to the POSIX operating system API, including the `close()` function for closing file descriptors (like sockets).
- **#include <arpa/inet.h>** : This header contains definitions for Internet operations, including functions like `inet_pton()` for converting IP addresses from text to binary form.

and `htons()` for host-to-network byte order conversion.

- `#include <sys/socket.h>` : This header provides core socket programming structures and functions, such as `socket()` for creating a socket and `sendto()` for sending data over a UDP socket.
- `#define PORT 5000` : Defines the port number on which the server is expected to be listening. This is a crucial identifier for the server application.
- `#define SERVER_IP "127.0.0.1"` : Defines the IP address of the server. `127.0.0.1` is the loopback address, meaning the server is expected to be running on the same machine as the client.
- `#define BUFFER_SIZE 1024` : Defines the maximum size of the buffer used to send and receive data. This limits the size of each log message that can be sent.

1.2. Main Function and Variable Declarations

Plain Text

```
int main() {  
  
    int sockfd;  
    char buffer[BUFFER_SIZE];  
    struct sockaddr_in server_addr;  
    socklen_t addr_len = sizeof(server_addr);
```

- `int main()` : This is the entry point of the C program.
- `int sockfd` : This variable will store the **socket file descriptor**. A socket file descriptor is an integer handle that the operating system uses to refer to a specific socket. It's similar to a file handle for network communication.
- `char buffer[BUFFER_SIZE]` : This declares a character array (buffer) of size `BUFFER_SIZE` (1024 bytes). It will be used to temporarily store the log messages entered by the user before sending them.
- `struct sockaddr_in server_addr` : This declares a structure of type `sockaddr_in`. This structure is used to hold **IPv4 socket address information**, including the IP address and port number of the server to which the client will send data.
- `socklen_t addr_len = sizeof(server_addr)` : This variable stores the size of the `server_addr` structure. It's used by socket functions to specify the size of the address structure.

1.3. Socket Creation

Plain Text

```

sockfd = socket(AF_INET, SOCK_DGRAM, 0);

if(sockfd < 0){
    perror("Socket creation failed");
    exit(EXIT_FAILURE);
}

printf("UDP Client started\n");

```

- `sockfd = socket(AF_INET, SOCK_DGRAM, 0);` : This is the core function call to create a new socket.
 - `AF_INET` : Specifies the **address family** as IPv4 (Internet Protocol version 4). This means the socket will use IPv4 addresses.
 - `SOCK_DGRAM` : Specifies the **socket type** as a datagram socket, which is used for UDP communication. UDP is a connectionless protocol, meaning data is sent in independent packets (datagrams) without establishing a persistent connection.
 - `0` : Specifies the **protocol**. When `0` is used, the operating system automatically chooses the appropriate protocol based on the address family and socket type. For `AF_INET` and `SOCK_DGRAM`, it defaults to UDP.
- `if(sockfd < 0)` : This checks if the `socket()` call failed. If it returns a negative value, an error occurred.
- `perror("Socket creation failed");` : If an error occurs, `perror()` prints a system error message along with the custom message "Socket creation failed".
- `exit(EXIT_FAILURE);` : Terminates the program with an error status.
- `printf("UDP Client started\n");` : Informs the user that the client has successfully started.

1.4. Server Address Configuration

Plain Text

```

memset(&server_addr, 0, sizeof(server_addr));

server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(PORT);
inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);

```

- `memset(&server_addr, 0, sizeof(server_addr));` : This function initializes the `server_addr` structure with zeros. It's good practice to clear the structure before filling it to avoid any

leftover or garbage values.

- `server_addr.sin_family = AF_INET;` : Sets the address family for the server's address to IPv4, matching the socket's address family.
- `server_addr.sin_port = htons(PORT);` : Sets the port number of the server. `htons()` (host to network short) converts the port number from the host's byte order to network byte order. This is essential because different computer architectures might store multi-byte values (like port numbers) in different byte orders, and network protocols require a consistent byte order.
- `inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);` : This function converts the IP address from its human-readable string format (`SERVER_IP` , e.g., "127.0.0.1") into a binary network address format, and stores it in the `sin_addr` field of the `server_addr` structure. `AF_INET` again specifies the IPv4 address family.

1.5. Main Communication Loop

Plain Text

```
while(1){

    printf("Enter log message: ");
    fgets(buffer, BUFFER_SIZE, stdin);

    buffer[strcspn(buffer, "\n")] =

    if(strcmp(buffer, "exit")==0)
        break;

    sendto(sockfd, buffer, strlen(buffer), 0,
            (struct sockaddr *)&server_addr, addr_len);

    printf("Log sent to server\n");
}
```

- `while(1)` : This creates an infinite loop, allowing the client to continuously send messages until explicitly told to stop.
- `printf("Enter log message: ");` : Prompts the user to enter a log message.
- `fgets(buffer, BUFFER_SIZE, stdin);` : Reads a line of text from standard input (`stdin`) and stores it in `buffer` . It reads up to `BUFFER_SIZE - 1` characters or until a newline character is encountered. The newline character is also stored in the buffer if there's space.
- `buffer[strcspn(buffer, "\n")] =` : This line is crucial for cleaning the input. `fgets()` includes the newline character (`\n`) if the user presses Enter. `strcspn()` finds the first occurrence

of `\n` in the `buffer`, and then that position is overwritten with a null terminator (`\0`). This effectively removes the newline character and properly null-terminates the string, which is important for `strlen()` and `strcmp()`.

- `if(strcmp(buffer,"exit")==0)` : Checks if the user entered the string "exit". `strcmp()` returns 0 if the strings are identical.
- `break;` : If the user types "exit", the loop terminates.
- `sendto(sockfd, buffer, strlen(buffer), 0, (struct sockaddr *)&server_addr, addr_len);` : This is the function call responsible for sending data over the UDP socket.
 - `sockfd` : The socket file descriptor created earlier.
 - `buffer` : The data to be sent (the user's log message).
 - `strlen(buffer)` : The number of bytes to send, which is the length of the string in the buffer (excluding the null terminator).
 - `0` : Flags for sending. Usually set to 0 for basic UDP.
 - `(struct sockaddr *)&server_addr` : A pointer to the `server_addr` structure, cast to a generic `struct sockaddr *`. This provides the destination IP address and port.
 - `addr_len` : The size of the `server_addr` structure.
- `printf("Log sent to server\n");` : Confirms to the user that the message has been sent.

1.6. Socket Closure

Plain Text

```
close(sockfd);  
  
return 0;  
}
```

- `close(sockfd);` : This function closes the socket file descriptor, releasing the resources associated with it. It's important to close sockets when they are no longer needed to prevent resource leaks.
- `return 0;` : Indicates that the program executed successfully.

2. Important Viva Questions and Answers

Q1: What is a server?

A1: In the context of computer networks, a **server** is a computer program or a device that provides functionality for other programs or devices, called "clients." This architecture is

known as the client-server model. Servers can provide various services, such as data storage, web pages, email, file sharing, or in this case, receiving log messages. A server typically listens for incoming requests from clients, processes them, and sends back responses.

Q2: What is UDP?

A2: UDP (User Datagram Protocol) is a connectionless, unreliable transport layer protocol. It is part of the Internet Protocol Suite, often referred to as UDP/IP. Key characteristics of UDP include:

- **Connectionless:** Unlike TCP, UDP does not establish a dedicated connection between the sender and receiver before sending data. Each datagram is sent independently.
- **Unreliable:** UDP does not guarantee delivery of datagrams, nor does it guarantee the order of delivery. Datagrams can be lost, duplicated, or arrive out of order without any notification to the sender.
- **No Flow Control/Congestion Control:** UDP does not implement mechanisms to manage the rate of data transmission or to prevent network congestion.
- **Lightweight:** Due to its simplicity and lack of overhead for connection management and reliability features, UDP is faster and more efficient for applications where speed is more critical than guaranteed delivery, such as streaming media, online gaming, and DNS lookups.

In this project, UDP is used because the client is sending log messages, where occasional loss of a message might be acceptable for the benefit of speed and lower overhead.

Q3: What are the roles and working of each component used in the code?

A3: Let's break down the key components and their roles:

- **sockfd (Socket File Descriptor):**
 - **Role:** An integer identifier representing the communication endpoint (the socket) created by the operating system. It's used by all subsequent socket-related functions to refer to this specific communication channel.
 - **Working:** Obtained by calling `socket()`. It acts as a handle for the client to send and receive data.
- **buffer (Character Array):**
 - **Role:** A temporary storage area for data. In this client, it holds the log message entered by the user before it's sent over the network.

- **Working:** `fgets()` reads user input into this buffer, and `sendto()` reads data from this buffer to transmit.
- **`struct sockaddr_in server_addr` (Server Address Structure):**
 - **Role:** Stores the complete network address (IP address and port number) of the destination server. This structure is specifically designed for IPv4 addresses.
 - **Working:** It's populated with the `AF_INET` family, the server's port (converted to network byte order), and the server's IP address (converted from string to binary format). This structure is passed to `sendto()` to specify where the datagram should be sent.
- **`socklen_t addr_len` (Address Length):**
 - **Role:** Stores the size of the `server_addr` structure. Many socket functions require the size of the address structure as an argument.
 - **Working:** Initialized with `sizeof(server_addr)` and passed to `sendto()`.
- **`socket()` function:**
 - **Role:** Creates a new communication endpoint (a socket).
 - **Working:** Takes arguments `AF_INET` (IPv4), `SOCK_DGRAM` (UDP), and `0` (default protocol) to create a UDP socket. It returns the `sockfd`.
- **`memset()` function:**
 - **Role:** Initializes a block of memory with a specified value.
 - **Working:** Used to clear the `server_addr` structure to all zeros before populating it with the server's address details, ensuring no garbage values interfere.
- **`htons()` function:**
 - **Role:** Converts a 16-bit (short) integer from host byte order to network byte order.
 - **Working:** Ensures that the `PORT` number is in the standard network byte order, which is big-endian. This is crucial for interoperability between different systems.
- **`inet_pton()` function:**
 - **Role:** Converts an IP address from its presentation (text) format to its numeric (binary) format.
 - **Working:** Takes the `SERVER_IP` string and converts it into the binary representation required for the `sin_addr` field of `server_addr`.
- **`fgets()` function:**
 - **Role:** Reads a line from the specified input stream.
 - **Working:** Reads the user's log message from `stdin` into the `buffer`.

- **strcspn() function:**
 - **Role:** Calculates the length of the initial segment of a string that consists entirely of characters *not* in another string.
 - **Working:** Used to find the position of the newline character (`\n`) in the `buffer` after `fgets()` , so it can be replaced with a null terminator (`\0`).
- **strcmp() function:**
 - **Role:** Compares two strings.
 - **Working:** Used to check if the user's input is "exit" to terminate the client program.
- **sendto() function:**
 - **Role:** Sends data over a socket to a specified destination address.
 - **Working:** Transmits the `buffer` content (log message) to the server identified by `server_addr` using the `sockfd` .
- **close() function:**
 - **Role:** Closes a file descriptor.
 - **Working:** Releases the resources held by the `sockfd` when the client is done communicating.

Q4: Why is `htons()` used for the port number and `inet_pton()` for the IP address?

A4:

- **htons() (Host to Network Short):** This function is used for the port number because network protocols, including IP and UDP, define a standard byte order for multi-byte values (like port numbers, which are 16-bit integers). This standard is called **network byte order** (big-endian). Different computer architectures (hosts) might use different internal byte orders (e.g., little-endian). `htons()` ensures that the port number is converted to the consistent network byte order before being placed into the `sockaddr_in` structure and sent over the network. This guarantees that the server, regardless of its architecture, will correctly interpret the port number.
- **inet_pton() (Internet Protocol to Numeric):** This function is used for the IP address because IP addresses are typically represented as human-readable strings (e.g., "127.0.0.1"). However, for network communication, the operating system and network hardware require the IP address in a compact, binary numeric format. `inet_pton()` converts this presentation format string into the appropriate binary representation that can be stored in the `in_addr_t` type within the `sockaddr_in` structure. This function is protocol-agnostic (supports both IPv4 and IPv6), specified by the `AF_INET` argument.

Q5: What is the significance of `AF_INET` and `SOCK_DGRAM` in the `socket()` call?

A5:

- **`AF_INET` (Address Family: Internet):** This argument specifies the **address family** that the socket will use. `AF_INET` indicates that the socket will communicate using **IPv4 addresses**. This means that the addresses used with this socket (like `SERVER_IP`) will be in the IPv4 format (e.g., `192.168.1.1`). If you were to use IPv6, you would specify `AF_INET6`.
- **`SOCK_DGRAM` (Socket Type: Datagram):** This argument specifies the **type of service** that the socket will provide. `SOCK_DGRAM` indicates that this is a **datagram socket**, which is used for **UDP (User Datagram Protocol)** communication. Datagram sockets provide a connectionless, unreliable service, where data is sent in independent packets (datagrams). If you wanted to use TCP (Transmission Control Protocol), which provides a connection-oriented, reliable service, you would specify `SOCK_STREAM`.

Together, `AF_INET` and `SOCK_DGRAM` tell the operating system to create a socket that uses IPv4 addresses and communicates using the UDP protocol.

Q6: Why is UDP considered "unreliable"?

A6: UDP is considered "unreliable" because it does not provide any mechanisms to guarantee the delivery, order, or integrity of data packets (datagrams). Specifically, UDP lacks:

- **Acknowledgement (ACK) and Retransmission:** The sender does not receive confirmation that a datagram has been received by the destination. If a datagram is lost in transit, the sender is unaware and does not retransmit it.
- **Sequencing:** UDP datagrams do not have sequence numbers. This means they can arrive at the destination out of order, and the receiver has no built-in way to reassemble them into the original sequence.
- **Flow Control:** There's no mechanism to prevent a fast sender from overwhelming a slow receiver.
- **Congestion Control:** UDP does not attempt to reduce its transmission rate when network congestion occurs, which can exacerbate congestion.
- **Error Checking (beyond checksum):** While UDP includes a checksum for basic error detection, it doesn't provide error correction or retransmission if an error is detected.

These characteristics make UDP suitable for applications where occasional data loss is acceptable or where the application layer handles reliability, and where low latency and high throughput are prioritized over guaranteed delivery.

Q7: What is the purpose of `perror()` and `exit(EXIT_FAILURE)` ?

A7:

- **`perror()`** : This function is used for **error reporting**. When a system call (like `socket()`) fails, it typically sets a global variable called `errno` to an error code. `perror()` takes a string argument (e.g., "Socket creation failed") and prints this string, followed by a colon, a space, and then a human-readable description of the error associated with the current value of `errno` . This is extremely helpful for debugging, as it provides specific information about why a system call failed.
- **`exit(EXIT_FAILURE)`** : This function is used for **program termination**. When `exit()` is called, the program immediately terminates. The argument `EXIT_FAILURE` is a macro defined in `<stdlib.h>` that typically expands to a non-zero integer value (often 1). By convention, a non-zero exit status indicates that the program terminated abnormally or encountered an error. This allows other programs or scripts that might execute this client to check its exit status and determine if it ran successfully or failed.

Together, they provide a robust way to handle critical errors: inform the user about the specific problem and then gracefully terminate the program, signaling failure to the operating system.

Q8: What is the role of `strcspn(buffer, "\n")` and why is it important?

A8:

- **Role:** `strcspn(buffer, "\n")` calculates the length of the initial segment of the string `buffer` that consists entirely of characters *not* found in the string `"\n"` (which just contains the newline character). In simpler terms, it finds the index of the first newline character (`\n`) in the `buffer` .
- **Importance:** When `fgets()` reads input from `stdin` , it includes the newline character (`\n`) that the user types when pressing Enter, provided there is enough space in the buffer. Network functions like `sendto()` and string manipulation functions like `strlen()` (which calculates the length of the string up to the null terminator) expect null-terminated strings without the trailing newline. If the newline character is not removed and replaced with a null terminator (`\0`), then:
 - `strlen()` would include the newline in its count, sending an extra, unwanted character.
 - The server might receive messages with trailing newlines, which could affect its parsing logic.
 - `strcmp()` might not work as expected if comparing a string with a newline to one without.

By replacing the `\n` with `\0` using `buffer[strcspn(buffer, "\n")] =` , the string is properly null-terminated, ensuring correct string length calculation and consistent data transmission.

Q9: Explain the client-server model in the context of this project.

A9: The client-server model is a distributed application architecture that partitions tasks or workloads between the providers of a resource or service, called **servers**, and service requesters, called **clients**.

In the context of this project:

- **Client (Your `server.c` code):**
 - **Role:** Initiates communication and requests services from the server. In this case, it's the program that the user runs to send log messages.
 - **Working:** It creates a UDP socket, configures the server's address, prompts the user for log messages, and then uses `sendto()` to transmit these messages as UDP datagrams to the server. It doesn't wait for a response from the server.
- **Server (Implied, but not provided in your code):**
 - **Role:** Listens for incoming requests from clients and provides services. In this scenario, it would be a separate program designed to receive and possibly process or store the log messages sent by your client.
 - **Working:** A UDP server would typically create a UDP socket, bind it to a specific IP address and port, and then enter a loop to continuously call `recvfrom()` to wait for and receive datagrams from clients. Upon receiving a datagram, it would extract the log message and perform its designated task (e.g., print to console, write to a file, store in a database).

This project demonstrates the client-side of this model, where your program acts as a client sending data to a (presumably existing) server.

Q10: What would happen if the server is not running when the client sends a message?

A10: Since UDP is a **connectionless** and **unreliable** protocol, the client will not receive an explicit error message directly from the `sendto()` call indicating that the server is not running. The `sendto()` function will likely succeed from the client's perspective, meaning the datagram was successfully handed over to the operating system's network stack for transmission.

However, the datagram will simply be dropped by the network if there is no process (the server) listening on the specified UDP port at the destination IP address. The client will not

be notified of this loss. The log message will effectively disappear without a trace from the client's point of view.

In some cases, if the destination host is reachable but no application is listening on the specific UDP port, the operating system on the *server machine* might send an **ICMP Port Unreachable** message back to the client. However, the client's UDP socket typically does not process these ICMP errors in a way that would cause `sendto()` to fail or provide an immediate error to the application. The client application would generally remain unaware of the server's absence unless it implements its own application-layer acknowledgment or timeout mechanisms.

This highlights UDP's unreliability: the sender has no guarantee that the data reached its destination or was processed.

Q11: How would the code change if you wanted to use TCP instead of UDP?

A11: Switching from UDP to TCP would involve significant changes, as TCP is a connection-oriented, reliable protocol. Here are the main differences:

1. **Socket Type:** The `socket()` call would change from `SOCK_DGRAM` to `SOCK_STREAM` :

Plain Text

```
sockfd = socket(AF_INET, SOCK_STREAM, 0); // SOCK_STREAM for TCP
```

2. **Connection Establishment:** Before sending data, the client would need to establish a connection with the server using `connect()` :

Plain Text

```
// After setting up server_addr
if (connect(sockfd, (struct sockaddr *)&server_addr, addr_len) < 0) {
    perror("Connection failed");
    exit(EXIT_FAILURE);
}
printf("Connected to server\n");
```

The `sendto()` function would no longer be used.

3. **Data Transmission:** Instead of `sendto()` , `send()` or `write()` would be used, as the destination is already established by `connect()` :

Plain Text

```
// Inside the loop
send(sockfd, buffer, strlen(buffer), 0); // Or write(sockfd, buffer, strlen(
```

4. **No Explicit Address in Send:** Because a connection is established, `send()` does not require the destination address as an argument.
5. **Server-Side Changes:** The server would also need to change significantly:
 - It would use `listen()` to mark the socket as a passive socket that will be used to accept incoming connection requests.
 - It would use `accept()` to accept a new connection from a client, which creates a *new* socket for that specific client communication.
 - It would use `recv()` or `read()` to receive data from the connected client.
6. **Reliability and Order:** TCP inherently provides guaranteed delivery, ordered data transfer, flow control, and congestion control, so the application wouldn't need to implement these features.

In summary, TCP requires a handshake (`connect()` on client, `listen()` / `accept()` on server) to establish a connection before data transfer, and then uses connection-specific `send()` / `recv()` calls, offering reliability at the cost of more overhead.

Q12: What is a port number and why is it important?

A12: A **port number** is a 16-bit unsigned integer (ranging from 0 to 65535) that identifies a specific process or application running on a network device (like a server or client). It works in conjunction with an IP address to form a complete network address, often called a **socket address** (IP address + port number).

Importance:

- **Process Identification:** While an IP address identifies a unique host on a network, a port number identifies a unique application or service *within* that host. For example, a web server typically listens on port 80 (HTTP) or 443 (HTTPS), an email server on port 25 (SMTP), and your log server on port 5000.
- **Multiplexing/Demultiplexing:** Port numbers allow a single IP address to host multiple network services simultaneously. When a packet arrives at an IP address, the operating system uses the port number in the packet's header to direct the data to the correct application process.
- **Communication Endpoint:** Together, the IP address and port number define a unique communication endpoint (a socket) in the network. This allows clients to send data to a specific service on a specific machine.

In your code, `PORT 5000` specifies that your client intends to send log messages to a server application that is listening for UDP datagrams on port 5000 on the `SERVER_IP` machine.

Q13: What is the loopback address `127.0.0.1` ?

A13: The IP address `127.0.0.1` is known as the **loopback address** or **localhost**. It is a special IP address reserved for referring to the current machine itself. Any data sent to `127.0.0.1` is routed back to the same machine, bypassing the network interface card and external network.

Importance/Use Cases:

- **Local Testing:** It's primarily used for testing network applications locally. Developers can run both a client and a server application on the same machine and have them communicate with each other using `127.0.0.1` without needing a physical network connection or worrying about external network configurations.
- **Development:** In development environments, it allows developers to test client-server interactions without deploying to a remote server.
- **Isolation:** Communication over the loopback interface is entirely internal to the machine, providing isolation from external network traffic.

In your project, `SERVER_IP "127.0.0.1"` means that your UDP client expects the corresponding UDP server to be running on the *same computer* where your client program is executed. This is a common setup for mini-projects and local development.

Q14: What is the difference between `printf()` and `perror()` ?

A14: Both `printf()` and `perror()` are functions used for outputting messages, but they serve different purposes and handle error reporting distinctly:

- **`printf()` (Print Formatted Output):**
 - **Purpose:** General-purpose output function used to print formatted text to the standard output stream (usually the console).
 - **Usage:** Takes a format string and a variable number of arguments. It allows for complex formatting of various data types.
 - **Error Handling:** Does not inherently handle system errors. If you want to report an error with `printf()`, you must manually construct the error message.
 - **Example:** `printf("Client started successfully.\n");` or `printf("Error code: %d\n", error_val);`
- **`perror()` (Print Error Message):**
 - **Purpose:** Specifically designed for **reporting system errors**. It provides a human-readable description of the last error that occurred during a system call or library

function call.

- **Usage:** Takes a single string argument, which is typically a custom message to precede the system error description. It automatically appends a colon, a space, and the system-defined error message corresponding to the global `errno` variable.
- **Error Handling:** Relies on the `errno` global variable, which is set by system calls upon failure. It's a convenient way to get detailed error information without needing to look up error codes manually.
- **Example:** `perror("Socket creation failed");` (If `errno` was `EACCES`, it might print: `Socket creation failed: Permission denied`)

In summary, `printf()` is for general informational output, while `perror()` is specifically for diagnosing and reporting system-level errors in a user-friendly way.

Q15: Why is `BUFFER_SIZE` defined as 1024?

A15: `BUFFER_SIZE` is defined as 1024 bytes (1 KB) to specify the maximum amount of data that can be stored in the `buffer` array at any given time. This size is chosen for several practical reasons:

- **Common Practice:** 1024 bytes (1 KB) is a very common and reasonable buffer size in network programming. It's large enough to accommodate typical short messages or commands without being excessively large.
- **Efficiency:** It's often a good balance between memory usage and the ability to handle a decent amount of data in a single read/write operation. Larger buffers can reduce the number of system calls but consume more memory, while smaller buffers might require more frequent system calls.
- **UDP Datagram Size:** While UDP datagrams can theoretically be much larger (up to 65,507 bytes), in practice, they are often kept smaller to avoid fragmentation at the IP layer. A common MTU (Maximum Transmission Unit) for Ethernet is 1500 bytes. Keeping the UDP payload well below this (e.g., 1024 bytes) helps ensure that the datagram fits into a single IP packet and avoids fragmentation, which can lead to performance degradation and increased packet loss.
- **Application Needs:** For sending log messages, 1024 bytes is usually more than sufficient for a single log entry.

Choosing an appropriate `BUFFER_SIZE` involves balancing memory consumption, network efficiency, and the specific requirements of the application.

References

- [1] Beej's Guide to Network Programming - Using Internet Sockets. (n.d.). Retrieved from <https://beej.us/guide/bgnet/>
- [2] Stevens, W. R., Fenner, B., & Rudoff, A. M. (2003). *UNIX Network Programming, Volume 1: The Sockets Networking API* (3rd ed.). Addison-Wesley Professional.
- [3] User Datagram Protocol (UDP). (n.d.). In *Wikipedia*. Retrieved from https://en.wikipedia.org/wiki/User_Datagram_Protocol
- [4] Loopback (networking). (n.d.). In *Wikipedia*. Retrieved from <https://en.wikipedia.org/wiki/Loopback>
- [5] Byte order. (n.d.). In *Wikipedia*. Retrieved from https://en.wikipedia.org/wiki/Byte_order